Requirement Document: N8N Hands-On Intermediate Course

Topic 1: Production Workflow Planning & Design Patterns

---

1. Document Control

Field	Value
Document Title	Topic 1: Production Workflow Planning & Design Patterns - Requirement File
Course	N8N - Hands On Intermediate Course (Level 2)
Target Audience	Automation Builders, System Integrators, QA Engineers
Version	1.0
Status	Approved
Last Updated	June 15, 2026
Prepared By	Lead Automation Architect

---

2. Learning Concepts

2.1 Production-Grade Automation Planning

Before implementing any business automation, experienced builders design a blueprint. Moving a workflow from sandbox to production requires transitioning from a “happy path” implementation to a resilient system capable of handling API latency, rate limits, schema changes, and service downtime.

2.2 Workflow Architecture Layers

A standard production workflow should follow a decoupled architecture consisting of four layers:

1. **Trigger Layer:** Ingestion of incoming webhooks, schedules, or events.
2. **Processing/Business Logic Layer:** Data transformation, enrichment, deduplication, and routing.
3. **Recovery/Error Handling Layer:** Retry configurations, dead-letter routing, and manual intervention queues.
4. **Notification Layer:** Alerting stakeholders (via Slack, Teams, or Email) when recovery steps fail.

2.3 Input and Output (I/O) Validation

Unvalidated inputs are the leading cause of production runtime errors. Schema enforcement checks whether the incoming payload contains all required keys (e.g., `customerEmail`, `transactionId`, `amount`, `currency`) and verifies data types and formats (e.g., regex matching for email patterns, positive number checks). Output validation ensures that responses sent to calling clients or external APIs meet their schema constraints.

2.4 Failure Points & Recovery Strategies

- **API Rate Limits (HTTP 429):** Mitigated by configuring retry mechanisms with exponential backoff and jitter.
- **Transient Network Failures (HTTP 5xx):** Managed by automatic retry settings.
- **Fatal Validation/Auth Failures (HTTP 4xx):** Bypasses retry queues and directs immediately to a manual intervention / logging queue.
- **Idempotency Risks:** Duplicate triggers are filtered out using unique event identifiers (e.g., UUID or `transactionId`) checked against a deduplication cache/store.

---

3. Hands-on Concepts with Syntax & Examples

3.1 Schema Ingestion & Type Enforcement

Using a JavaScript code block to validate incoming payload headers and values:

```
// Input validation using n8n JavaScript Code Node
const payload = $json.payload || {};
const errors = [];

// 1. Email validation using standard RFC 5322 regex
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!payload.customerEmail || !emailRegex.test(payload.customerEmail)) {
  errors.push("Invalid or missing 'customerEmail'");
}

// 2. Amount validation (must be positive numerical value)
```

```
const amount = Number(payload.amount);
if (isNaN(amount) || amount <= 0) {
  errors.push("Field 'amount' must be a positive number greater than 0");
}

// 3. Currency validation (ISO 4217: 3 uppercase letters)
const currencyRegex = /^[A-Z]{3}$/;
if (!payload.currency || !currencyRegex.test(payload.currency)) {
  errors.push("Field 'currency' must be a valid 3-letter uppercase ISO code (e.g. USD, EUR, GBP)");
}

const isValid = errors.length === 0;

return {
  json: {
    transactionId: $json.transactionId,
    isValid: isValid,
    validationErrors: errors,
    sanitizedPayload: {
      transactionId: $json.transactionId,
      customerEmail: String(payload.customerEmail || '').toLowerCase().trim(),
      amount: amount,
      currency: String(payload.currency || '').toUpperCase().trim(),
      description: payload.description || 'Standard Order Charge'
    }
  }
};
```

3.2 Error Routing Settings

In n8n, every node has configuration settings for error handling. For production, the following standard parameters must be applied to critical HTTP requests:

- **Retry On Failure:** Enabled (`true`)
- **Max Retries:** 3
- **Delay Between Retries (ms):** 5000 (5 seconds)
- **Backoff Policy:** Exponential / Standard

---

4. Business Requirements Document (BRD)

4.1 Business Objectives

To establish a production-grade framework for business automations that reduces service downtime, eliminates duplicate transaction processing, and provides automated auditing and troubleshooting mechanisms.

4.2 Business Requirements Table

ID	Business Requirement	Business Rationale	Priority
BRD-001	Workflows shall handle transient system downtime without manual developer intervention.	Reduces maintenance overhead and prevents transaction drops.	High
BRD-002	The system shall prevent duplicate executions of identical triggers (Idempotency).	Avoids double-billing or duplicate charges to customers.	High
BRD-003	All workflow execution metrics and failure occurrences shall be logged for analysis.	Necessary for operations tracking and SLA verification.	Medium

---

5. Product Requirements Document (PRD)

5.1 Product Capabilities

ID	Product Requirement	User Value	Priority
PRD-001	Automated Retry Mechanism	User transactions automatically re-attempt if the server is temporarily busy.	High
PRD-002	Centralized Incident Dashboard	Operations team can view failed runs, payloads, and retry history in one UI.	High
PRD-003	Real-Time Failure Notification	Teams get real-time alerts with direct execution links to speed up troubleshooting.	Medium

---

6. Software Requirements Specification (SRS) / Technical Requirements

6.1 Functional Requirements

* **FR-001:** The system shall validate incoming payloads against a JSON schema prior to triggering the business logic layer.

* **FR-002:** The system shall route any node failure or unhandled exception to a centralized error workflow listener.
* **FR-003:** The system shall check an in-memory cache, Redis cache, or relational database table for the unique trigger key `transactionId` to verify if the event has already been processed.

6.2 Non-Functional Requirements

* **NFR-001 (Resilience):** The workflow shall recover from API timeouts by retrying up to 3 times with a 5000 ms delay.
* **NFR-002 (Security):** All API authorization keys used by the integration workflow shall be loaded dynamically from credential managers and never hardcoded in node parameters.

---

7. User Stories & Acceptance Criteria

US-001: Automatic Transient Failure Recovery

* **User Story:** As an Operations Manager, I want the automation workflow to retry transient API connection failures automatically, so that I do not have to manually re-run failed workflows for temporary network drops.
* **Acceptance Criteria:**
  * Given an external API returns a 503 Service Unavailable error,
  * When a workflow node attempts to send data to it,
  * Then the workflow must wait for 5 seconds and retry the request up to 3 times before declaring a hard failure.

US-002: Real-time Incident Notification

* **User Story:** As a Support Engineer, I want to receive an instant alert when a workflow encounters a fatal error, so that I can fix the issue within our SLA.
* **Acceptance Criteria:**
  * Given a workflow execution encounters an unhandled exception or exhausted retries,
  * When all retry options are exhausted or bypassed,
  * Then the system must compile incident details containing the Workflow Name, Failed Node, Error Message, Timestamp, and Execution URL.

---

8. Test Cases

ID	Test Case	Preconditions	Steps	Expected Result	Priority	Linked Req
TC-001	Valid Payload Ingestion	Webhook endpoint is active.	Send valid JSON request containing all fields.	Webhook returns 202 Accepted, passes validation, calls Billing API, and logs PROCESSED audit record.	High	FR-001, BRD-001

TC-002	Idempotency Bypass Check	TC-001 has been executed.	Send identical request with same `transactionId` a second time.	`Code_CheckIdempotency` detects duplicate, routes to `Log_BypassDuplicate`, and halts.	High	BRD-002, FR-003
TC-003	Schema Validation Rejection	Webhook endpoint is active.	Send JSON request with invalid email and negative amount.	`Code_ValidateSchema` fails, routes to `Log_ValidationError` with error details, and halts.	High	FR-001
TC-004	Transient Error Retry & Incident Alert	Target server returns HTTP 503 error.	Trigger workflow that sends request to target.	Node retries 3 times with 5000ms delay, then routes unhandled failure to `Error_Trigger` and logs alert.	High	US-001, FR-002

---

9. Assumptions, Dependencies, and Risks

9.1 Assumptions

* ASM-001: The incoming webhook payload has a stable key structure, or any schema version changes will be updated in the validation node.
* ASM-002: The automation engine has access to the target alerting Slack/Email workspace webhook URI or incident handler.

9.2 Dependencies

* DEP-001: n8n instance version 1.0.0+ with active database connection.
* DEP-002: External target service has a maximum response timeout threshold of 30 seconds.

9.3 Risks

* RSK-001 (Infinite Loops / Overload): A misconfigured retry schedule could create resource starvation in the engine.
Mitigation:* Cap retry attempts strictly at 3 with 5000ms delay and exponential backoff.
* RSK-002 (Memory Bloat in Cache): Storing unlimited transaction IDs in memory causes OOM crash.
Mitigation:* Evict oldest IDs beyond 500 records using `.shift()` array limits.

---

10. Glossary

Term	Definition

n8n	An extendable node-based workflow automation tool used to connect APIs and handle complex data routing.
Idempotency	The property of a system where an operation can be applied multiple times without changing the result beyond the initial application.
Exponential Backoff	An algorithm that increases the waiting time between retries exponentially to prevent overloading target servers.
Dead-Letter Queue (DLQ)	A service queue designed to hold messages that cannot be processed successfully due to system errors or bad formatting.